

NEW SERIES

THE INVISIBLE HIGHWAY
THAT CONNECTS EVERYTHING

Before Cloud,
Before DevOps,
Before Everything...
There is NETWORKING!

NETWORKING

THE **COMPLETE** ROADMAP
FROM **BASICS** TO **ADVANCED**



YOU



NETWORK



SERVERS



EVERY CLICK. EVERY REQUEST. EVERY CONNECTION.
HAPPENS BECAUSE OF NETWORKING.

IN THIS SERIES, WE WILL COVER:

- ✓ Fundamentals
- ✓ Protocols
- ✓ IP Addressing & Subnetting
- ✓ Ports & Sockets
- ✓ Routing, Switching & VLAN
- ✓ Security
- ✓ Troubleshooting
- ✓ Real-world Examples & More

“

You don't need to
be a Network Engineer
to understand **Networking**,
But you must understand
it to grow in **Tech**.

”



LET'S BUILD A
STRONG FOUNDATION
TOGETHER!



LEARN. PRACTICE. BUILD. GROW.

@NeerajSingh-DevOps



② WHAT IS NETWORKING?

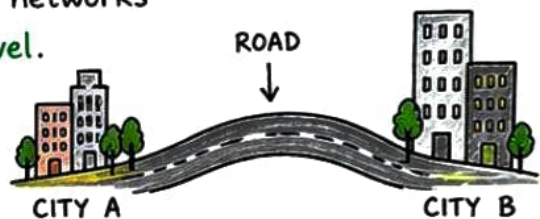
At its core,
Networking is
all about
CONNECTION!

★ DEFINITION :

Networking is the process of **connecting** two or more devices together so that they can **communicate**, **share data** and **resources**.

★ REAL LIFE ANALOGY :

Just like **roads** connect different **places** so that people, vehicles and goods can **travel**, networks connect devices so that **data** can **travel**.



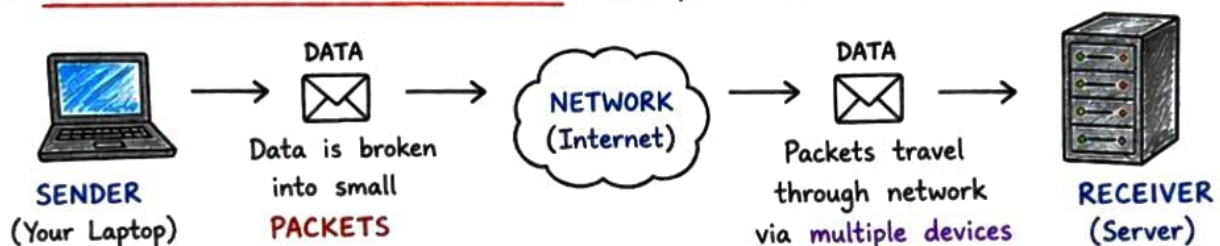
★ WHY DO WE NEED NETWORKS?

- To **communicate** (Email, Chat, Video Calls)
- To **share files**, printers and resources
- To access **information** and services
- To enable **collaboration**
- To make our work **faster**, **smarter** and **easier**
- To connect the **world** together 🌍

EXAMPLES

- Browsing Websites
 - Online Banking
 - Streaming Movies
 - Online Classes
 - Cloud Services
- ... all possible because
of **NETWORKS!**

★ HOW DOES NETWORKING WORK? (Simple View)



At the receiver end, packets are **reassembled** to form the **original data**.



KEY TAKEAWAY

Networking is the **backbone** of the digital world.
Without networking, there is **no internet**,
no communication and **no modern technology**. 😊

BY

@NeerajSingh-DevOps



3

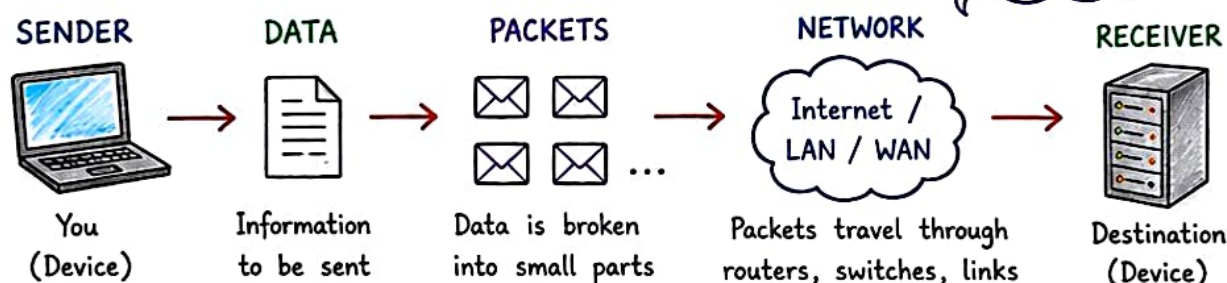
HOW COMMUNICATION HAPPENS

THE BIG PICTURE

When two devices communicate, data travels in a **step-by-step** journey.

★ Think:

Networks are like a post office for data!



At the receiver end, packets are **reassembled** in the correct order to form the **original data**.

WHAT IS A PACKET?

A **packet** is a small unit of data that carries:

- Source Address
- Destination Address
- Data (Payload)
- **Control Information** (for routing, ordering, error checking)

PACKET ANATOMY (Simplified)

Source Address	Destination Address	Control Info	Data (Payload)
↓	↓	↓	↓
Where it comes from	Where it needs to go	Helps in delivery	Actual Information

WHY PACKETS?

- Breaks large data into small parts.
- Easier and faster to transmit.
- Takes multiple paths for reliability.
- If one packet is **lost**, others can still reach.
- Helps in **efficient** use of network.

★ If the road is blocked, packets can take **another route**!

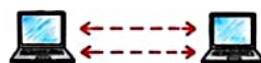


TYPES OF CONNECTIONS

- **Connection-Oriented** (Reliable)
 - A dedicated **connection** is established first.
 - Data is delivered in order.
 - Example: TCP (Used in web, email, file transfer)



- **Connectionless** (Faster)
 - No connection setup.
 - Faster but less reliable.
 - Example: UDP (Used in gaming, streaming, video calls)



KEY TAKEAWAY:

Data → Packets → Network → **Reassembly** → Original Data
 This is the foundation of all networking!



4

TYPES OF NETWORKS

Networks are of different types based on their **coverage area** and **purpose**.

★ Different needs,
Different sizes,
One Goal →
CONNECTION!



1 PAN (Personal Area Network)

- Covers a very small area (around a person).
- Used to connect personal devices.
- Range: 1 to 10 meters.

Example:

Bluetooth between
Mobile and Earbuds

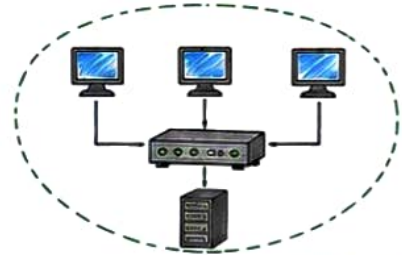


2 LAN (Local Area Network)

- Covers a small area like a room, building or campus.
- High speed, privately owned.
- Used in homes, offices, schools.

Example:

Network in an
office building

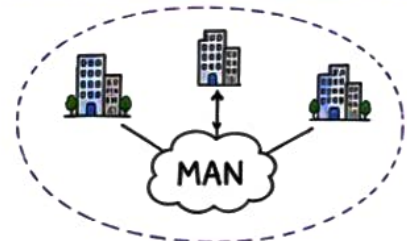


3 MAN (Metropolitan Area Network)

- Covers a city or large campus.
- Connects multiple LANs.
- Managed by an organization or service provider.

Example:

Cable TV network
in a city



4 WAN (Wide Area Network)

- Covers a large geographical area like country, continent or world.
- Connects MANs or LANs.
- Internet is the biggest WAN.

Example:

The Internet
(Global Network)



5 CAN (Campus Area Network)

- Covers multiple buildings within a campus (college, company, etc).
- Bigger than LAN but smaller than MAN.

Example:

University
Campus Network



QUICK COMPARISON

Type	Coverage	Range (Approx)	Example
PAN	Very Small (Personal)	1 - 10 meters	Bluetooth, Smartwatch
LAN	Small (Room/Building)	Up to few km	Office, Home Network
MAN	City Level	10 - 50 km	Cable TV Network, City Network
WAN	Country/Continent/World	100s - 1000s km	Internet
CAN	Campus Level	1 - 10 km	College, Corporate Campus

★ From small
connections
to global
networks,
everything is
connected!

5 NETWORK DEVICES

Networking devices are the building blocks that help devices **connect**, **communicate** and **share data**.

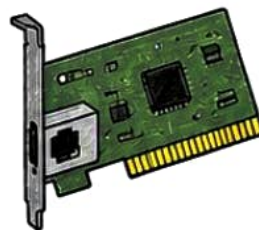
★ Think of them as the **traffic controllers** of the network!



1 NIC (Network Interface Card)

Allows a device (like your laptop or server) to **connect** to a network.

- Has a unique **MAC address**.
- Converts **data** between device and network.



Used in:

Every device that connects to a network.

2 HUB (Repeater)

Connects multiple devices and **broadcasts** data to all devices.

- Works at **Physical Layer** (Layer 1).
- Not intelligent, **slow** and rarely used now.



Used in:

Old networks (now obsolete).

3 SWITCH (Multiport Bridge)

Connects devices in a LAN and **sends data** only to the intended device.

- Works at **Data Link Layer** (Layer 2).
- Improves speed and security.



Used in:

Modern LANs, offices, homes, data centers.

4 ROUTER

Connects different networks and finds the **best path** to send data.

- Works at **Network Layer** (Layer 3).
- Uses **IP address** to route data.



Used in:

Home Wi-Fi, offices, ISPs, internet access.

5 MODEM (Modulator - Demodulator)

Connects your network to the ISP (Internet Service Provider) and **converts signals**.

- Modulates **digital** → **analog** (sending)
- Demodulates **analog** → **digital** (receiving)



Used in:

Connecting to ISP / Internet.



KEY TAKEAWAY

- ✓ NIC connects a device to the network.
- ✓ HUB shares data with all devices.
- ✓ SWITCH sends data only where it's needed.
- ✓ ROUTER connects different networks.
- ✓ MODEM connects us to the Internet.

TRAFFIC ANALOGY

	NIC	Vehicle enters the road
	HUB	Announces to everyone (all lanes)
	SWITCH	Sends vehicle to exact lane
	ROUTER	Chooses best route to destination
	MODEM	Connects to highway (Internet)

6 THE OSI MODEL

Open Systems Interconnection Model

The OSI Model is a **7-layer** framework that helps us understand how data travels from one device to another over a network.

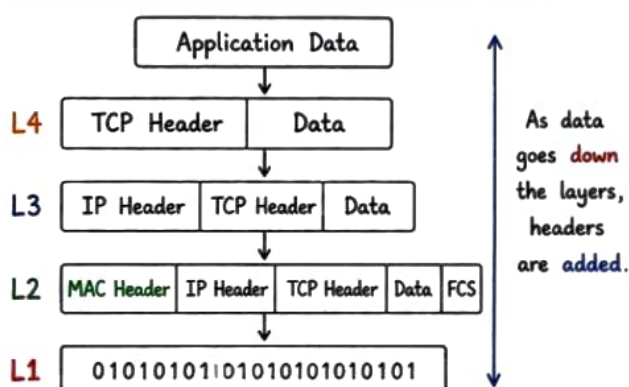
★ It breaks down network communication into **7 logical layers**. Each layer has a specific job!



THE 7 LAYERS OF OSI MODEL

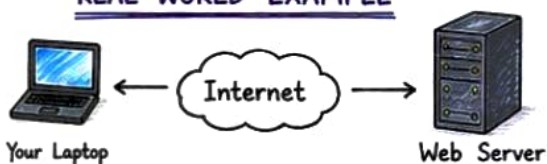
Layer	Layer Name	What it does	Examples / Protocols	PDU
7	APPLICATION (Layer 7) 	Provides network services directly to user applications.	HTTP, HTTPS, FTP, SMTP, DNS, DHCP, SSH	Data
6	PRESENTATION (Layer 6) 	Handles data formatting, encryption, decryption, and compression.	SSL/TLS, JPEG, MPEG, ASCII, Encryption	Data
5	SESSION (Layer 5) 	Establishes, manages and terminates sessions between applications.	NetBIOS, RPC, SQL, Named Pipes	Data
4	TRANSPORT (Layer 4) 	Provides end-to-end communication, segmentation, reliability (TCP), flow control.	TCP, UDP	Segment (TCP) Datagram (UDP)
3	NETWORK (Layer 3) 	Handles logical addressing and routing of packets across networks.	IP (IPv4, IPv6), ICMP, IGMP	Packet
2	DATA LINK (Layer 2) 	Provides node-to-node delivery, framing, MAC addressing, and error detection.	Ethernet, PPP, MAC, VLAN, ARP	Frame
1	PHYSICAL (Layer 1) 	Transmits raw bits over the physical medium (cables, signals).	Ethernet (PHY), Hubs, Repeaters, Cables	Bits

HOW DATA TRAVELS (ENCAPSULATION)



At the receiver, the process is reversed (**DECAPSULATION**) and the original data is delivered to the application.

REAL WORLD EXAMPLE



- ① You open a browser (Application Layer) and request a webpage.
- ② The data is formatted and encrypted.
- ③ A session is created to talk to the server.
- ④ TCP breaks data into segments.
- ⑤ IP finds the best path to the server.
- ⑥ Data Link sends it as frames to next device.
- ⑦ Physical layer sends bits as electrical/optical signals over cables or Wi-Fi.



The OSI Model is a reference model. In real life, **TCP/IP model** is used in the Internet.

7

TCP/IP MODEL

The TCP/IP model is a **4-layer model** used in **real** networks and the **Internet**.

★ It is the foundation of the **Internet!**



THE 4 LAYERS OF TCP/IP MODEL

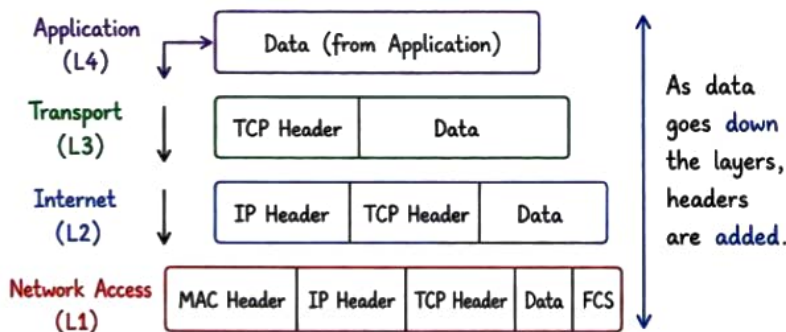
Layer	Purpose	Example Protocols		PDU
4	APPLICATION LAYER 	Provides network services directly to user applications.	HTTP, HTTPS, FTP, SMTP, DNS, DHCP, SSH, SNMP	Data
3	TRANSPORT LAYER 	Provides end-to-end communication, reliability and flow control.	TCP, UDP	Segment (TCP) Datagram (UDP)
2	INTERNET LAYER 	Handles logical addressing and routing of packets.	IP (IPv4, IPv6), ICMP, IGMP	Packet
1	NETWORK ACCESS LAYER 	Transmits data on the physical network. Handles framing and MAC addressing.	Ethernet, ARP, PPP, VLAN	Frame

KEY POINTS



- ★ TCP = Reliable (Connection-oriented)
- ★ UDP = Fast (Connectionless)
- ★ IP = Handles addressing & routing
- ★ Lower layers deal with how data is sent, upper layers deal with what data is sent.

ENCAPSULATION (SENDING SIDE)



REMEMBER:

Each layer adds information (its header) to the data from the layer above it.

At the receiver, the process is reversed (DECAPSULATION) and original data is delivered to the application.

REAL WORLD EXAMPLE - OPENING A WEBSITE



- 1 You type a website address (example.com) in your browser.
- 2 DNS (Application Layer) converts the name to an IP address.
- 3 IP (Internet Layer) finds the best path and sends packets to the server.
- 4 Data is sent on the network as frames over cables or Wi-Fi.
- 5 Web server receives the request and sends back the website data.



TCP/IP Model is **Practical**. It shows how data actually travels in real networks and is used **everywhere** → from your phone to the entire **Internet!**



8

IP ADDRESSING

Every device on a network has a unique IP address, so it can **identify** and **communicate** with others.

★ No IP address,
No **Communication**!



① WHAT IS AN IP ADDRESS?

- IP (Internet Protocol) address is a **unique** address assigned to every device connected to a network.
- It helps in identifying the device and its location in the network.
- IP address works at **Network Layer** (Layer 3)

IP ADDRESS VERSION

IPv4

- 32-bit address
- Example: 192.168.1.1
- Written in dotted decimal notation
- Exhausting (Running out of IPs)

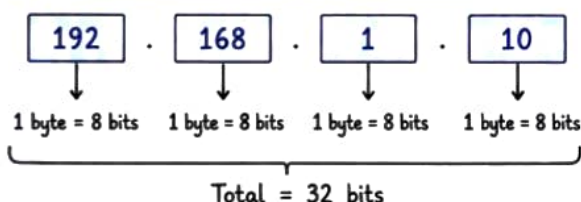
192.168.1.1

IPv6

- 128-bit address
- Example: 2001:0db8::1
- Written in hexadecimal notation
- Huge address space (Future-proof)

2001:0db8::1

② IP ADDRESS FORMAT (IPv4)



EXAMPLE

IP Address : 192.168.1.10

Binary : 11000000.10101000.00000001.00001010

Range : 0.0.0.0 to 255.255.255.255

③ TYPES OF IP ADDRESSES

Type	Range (First Octet)	Purpose	Example
Class A	1 - 126	Large Networks	10.0.0.1
Class B	128 - 191	Medium Networks	172.16.0.1
Class C	192 - 223	Small Networks	192.168.1.1
Class D	224 - 239	Multicast	224.0.0.1
Class E	240 - 255	Experimental	240.0.0.1

SPECIAL IP ADDRESSES

- 127.0.0.1 → Loopback (Localhost)
- 0.0.0.0 → "This" Network
- 255.255.255.255 → Broadcast



IP address + **Subnet Mask** helps in identifying Network and Host portion.

④ PUBLIC vs PRIVATE IP

PUBLIC IP

- Assigned by ISP
- Unique on the Internet
- Reachable from anywhere
- Used to access your network



PRIVATE IP

- Assigned within a private network
- Not unique on the Internet
- Not reachable from outside
- Used inside LAN



⑤ PRIVATE IP RANGES

Class	Private IP Range
Class A	10.0.0.0 - 10.255.255.255
Class B	172.16.0.0 - 172.31.255.255
Class C	192.168.0.0 - 192.168.255.255



KEY TAKEAWAY

- ✓ IP address is the identity of a device on a network.
- ✓ IPv4 uses 32-bit, IPv6 uses 128-bit addressing.
- ✓ Private IPs are used inside networks, Public IPs on the Internet.
- ✓ IP addressing is the foundation of all network communication.



The right IP, at the right place, makes **communication** possible!

9 SUBNETTING

Subnetting is the process of dividing a large network into smaller networks to improve performance, security and management.

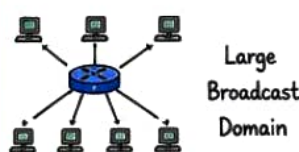
★ Subnetting helps us divide a network into smaller networks (SUBNETS)!



1 WHY SUBNETTING?

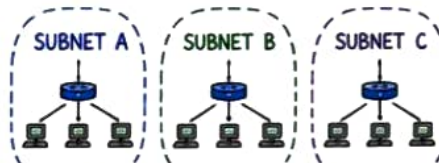
- Efficient use of IP addresses
- Reduces broadcast traffic
- Improves security
- Better network performance
- Easier network management

WITHOUT SUBNETTING



Large Broadcast Domain

WITH SUBNETTING



Smaller Broadcast Domains

2 SUBNETTING BASICS

- A subnet is created by borrowing bits from the HOST portion.
- More borrowed bits → More subnets but fewer hosts per subnet.
- Use Subnet Mask to define the network and host parts.

EXAMPLE

Network: 192.168.1.0/24

/24 means 24 bits for network and 8 bits for hosts.

Network (24 bits)

Host (8 bits)

11000000.10101000.00000001.00000000

Subnet Mask: 255.255.255.0

3 SUBNET MASK QUICK REFERENCE

Prefix	Subnet Mask	Network Bits	Host Bits	Total IPs	Usable Hosts
/24	255.255.255.0	24	8	256	254
/25	255.255.255.128	25	7	128	126
/26	255.255.255.192	26	6	64	62
/27	255.255.255.224	27	5	32	30
/28	255.255.255.240	28	4	16	14
/29	255.255.255.248	29	3	8	6
/30	255.255.255.252	30	2	4	2

FORMULA

- Total IPs = 2^n (where n = host bits)
- Usable Hosts = Total IPs - 2 (Network ID and Broadcast ID are not usable)

4 SUBNETTING EXAMPLE

Given Network: 192.168.1.0/24

Need: 4 Subnets

Solution:

- 4 subnets require 2 bits ($2^2 = 4$ subnets)
- New prefix = /24 + 2 = /26
- New Subnet Mask = 255.255.255.192

Subnet	Network ID	First Usable IP	Last Usable IP	Broadcast ID
1	192.168.1.0	192.168.1.1	192.168.1.62	192.168.1.63
2	192.168.1.64	192.168.1.65	192.168.1.126	192.168.1.127
3	192.168.1.128	192.168.1.129	192.168.1.190	192.168.1.191
4	192.168.1.192	192.168.1.193	192.168.1.254	192.168.1.255



KEY TAKEAWAY

- ✓ Subnetting breaks a big network into smaller parts.
- ✓ It saves IP addresses and improves performance & security.
- ✓ Understand subnet mask, prefix and ranges.



The better you subnet, the better your network runs!

10

CIDR (CLASSLESS INTER-DOMAIN ROUTING)

CIDR is a method of allocating IP addresses and routing that **reduces wastage** and improves efficiency.

★ CIDR gives us flexibility and efficient use of IP addresses!



1 WHAT IS CIDR?

- CIDR allows **variable length** subnet masks instead of fixed class-based masks.
- Written in the format: IP Address / Prefix
- Example: 192.168.1.0/24

CIDR NOTATION

IP Address Prefix Length
↓ ↓
192.168.1.0/24

/24 means first 24 bits are the network and remaining 8 bits are for hosts.



QUICK FACTS

- Default CIDR for Class A = /8
- Default CIDR for Class B = /16
- Default CIDR for Class C = /24

2 PREFIX LENGTH REFERENCE

Prefix	Subnet Mask	Network Bits	Host Bits	Total IPs	Usable Hosts
/8	255.0.0.0	8	24	16,777,216	16,777,214
/16	255.255.0.0	16	16	65,536	65,534
/24	255.255.255.0	24	8	256	254
/25	255.255.255.128	25	7	128	126
/26	255.255.255.192	26	6	64	62
/27	255.255.255.224	27	5	32	30
/28	255.255.255.240	28	4	16	14
/29	255.255.255.248	29	3	8	6
/30	255.255.255.252	30	2	4	2



FORMULAS

- Total IPs = 2^n
(where n = host bits)
- Usable Hosts = Total IPs - 2
(Network ID and Broadcast ID are not usable)

3 NETWORK RANGE IN CIDR

Example: 192.168.10.0/24

Network (24 bits) Host (8 bits)

11000000 . 10101000 . 00001010 . 00000000

- Network Address : 192.168.10.0
- First Usable IP : 192.168.10.1
- Last Usable IP : 192.168.10.254
- Broadcast Address : 192.168.10.255
- Subnet Mask : 255.255.255.0 (/24)

4 CIDR vs CLASSFUL

Feature	Classful	CIDR
Addressing	Fixed (A, B, C)	Variable length
Wastage	High	Low
Routing	Less efficient	More efficient
Flexibility	Limited	High
Example	192.168.1.0 (Class C)	192.168.1.0/27

5 CIDR SUBNETTING EXAMPLE

Given Network: 192.168.1.0/24

Need: 4 Subnets

Solution:

- 4 subnets require 2 bits ($2^2 = 4$)
- New prefix = /24 + 2 = /26
- New Subnet Mask = 255.255.255.192

Subnet	Network ID	First Usable IP	Last Usable IP	Broadcast ID
1	192.168.1.0	192.168.1.1	192.168.1.62	192.168.1.63
2	192.168.1.64	192.168.1.65	192.168.1.126	192.168.1.127
3	192.168.1.128	192.168.1.129	192.168.1.190	192.168.1.191
4	192.168.1.192	192.168.1.193	192.168.1.254	192.168.1.255



KEY TAKEAWAY

- ✓ CIDR helps us use IP addresses efficiently.
- ✓ It reduces wastage and improves routing performance.
- ✓ Understand prefix, subnet mask, and ranges for better network design.

Plan your subnets well, and your network will **scale** and perform like a **pro**!



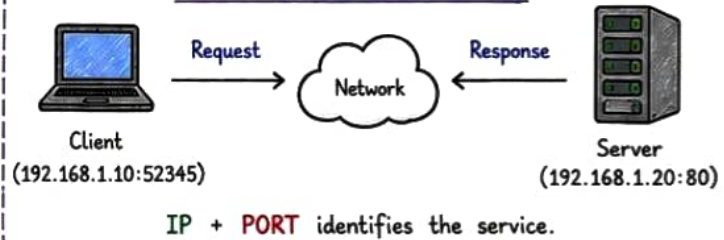
★ Ports are like **doors** on your system. They allow specific types of traffic in and out.



① WHAT IS A PORT?

- A port is a logical **endpoint** used by operating systems to identify a specific process or service.
- Network communication uses **IP address + Port number** to deliver data to the correct application.

HOW COMMUNICATION WORKS



② PORT NUMBER RANGE

Range	Name	Description
0 - 1023	Well Known Ports	Used by common services (such as HTTP, SSH, DNS).
1024 - 49151	Registered Ports	Used by user applications and services.
49152 - 65535	Dynamic / Private Ports	Used for temporary connections (ephemeral).

💡 **Note:** Ports below **1024** usually require root / administrator privileges to use.

④ TCP vs UDP PORTS

TCP	UDP
- Connection-oriented - Reliable (acknowledged) - Slower but accurate - Used by: HTTP, HTTPS, SSH, FTP, SMTP, MySQL	- Connectionless - Faster but not reliable - No guarantee of delivery - Used by: DNS, DHCP, Streaming, VoIP, Gaming

⑤ CHECK OPEN PORTS (LINUX)

1) Using netstat (older)

```

sudo netstat -tulnp
# -t: TCP, -u: UDP, -l: listening,
# -n: numeric, -p: process
  
```

2) Using ss (recommended)

```

sudo ss -tulnp
# Faster and modern alternative
  
```

3) Using lsof

```

sudo lsof -i -P -n
# Shows processes using ports
  
```

③ COMMONLY USED PORTS

Port	Protocol	Service	Description
20	TCP	FTP (Data)	File Transfer (Data)
21	TCP	FTP (Control)	File Transfer (Control)
22	TCP	SSH	Secure Shell
23	TCP	Telnet	Remote Login
25	TCP	SMTP	Email Sending
53	UDP/TCP	DNS	Domain Name System
80	TCP	HTTP	Web Traffic (Unsecured)
443	TCP	HTTPS	Web Traffic (Secure)
3306	TCP	MySQL	MySQL Database
5432	TCP	PostgreSQL	PostgreSQL Database
6379	TCP	Redis	Redis Cache
8080	TCP	HTTP-Alt	Alternative HTTP

⑥ FIREWALL & PORTS

- Firewall controls which ports are allowed or blocked.
- Only open the ports that are required.
- Example (UFW - Ubuntu Firewall):

```

sudo ufw allow 22      # Allow SSH
sudo ufw allow 80      # Allow HTTP
sudo ufw allow 443     # Allow HTTPS
sudo ufw deny 23       # Block Telnet
sudo ufw status        # Check status
  
```



KEY TAKEAWAY

- ✓ Ports identify services on a device.
- ✓ Use the right port for the right service.
- ✓ Know the port ranges and common ports.
- ✓ Always **secure** your system by closing unnecessary ports!



Remember: **IP address** finds the device,
Port number finds the right service on that device.

Your App → IP Address → Port Number → Service

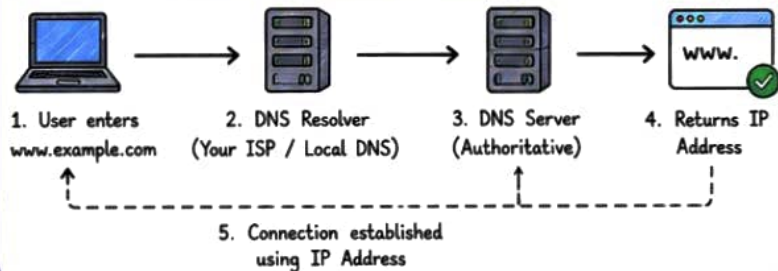
DNS translates human-readable names into IP addresses.



1 WHAT IS DNS?

- DNS is like the Internet's phonebook.
- It converts domain names (e.g., google.com) into IP addresses (e.g., 142.250.190.14).
- Makes it easy for users to access websites without remembering IPs.

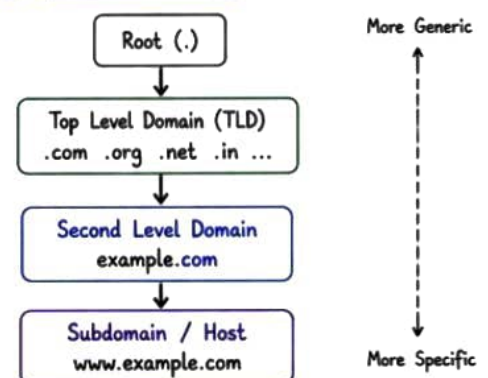
HOW DNS WORKS (HIGH LEVEL)



2 DNS RECORD TYPES

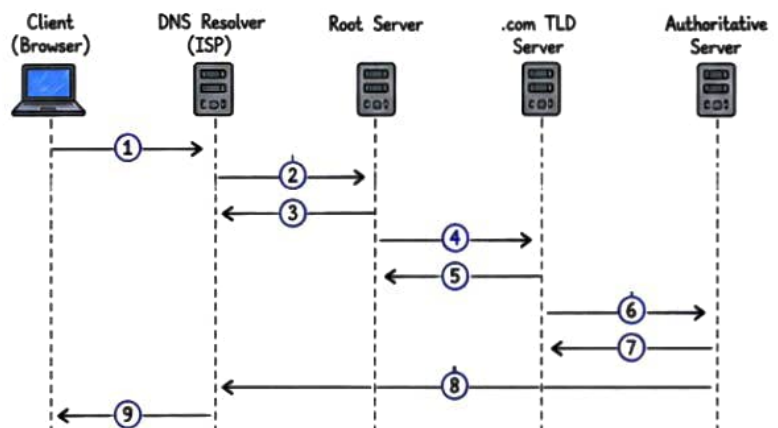
Record Type	Purpose	Example
A	Maps domain to IPv4 address	example.com → 93.184.216.34
AAAA	Maps domain to IPv6 address	example.com → 2606:2800:220:1:248:1893:25C8:1946
CNAME	Alias of one domain to another	www.example.com → example.com
MX	Mail server for the domain	example.com → mail.example.com
NS	Name servers for the domain	example.com → ns1.example.com
TXT	Text information (SPF, verification)	example.com → "v=spf1 include:_spf.google.com ~all"

3 DNS HIERARCHY



4 DOMAIN NAME RESOLUTION EXAMPLE

1. User types: www.example.com in browser
2. Browser checks local cache
3. If not found, query goes to DNS Resolver
4. Resolver queries Root server
5. Root server replies with .com TLD server
6. Resolver queries .com TLD server
7. TLD server replies with example.com authoritative DNS server
8. Resolver queries authoritative server
9. Authoritative server returns IP address
10. Resolver returns IP to browser
11. Browser connects to the IP



5 COMMON DNS COMMANDS (LINUX)

```
# Check DNS resolution
nslookup example.com
dig example.com

# Get A record
host example.com

# Reverse lookup (IP to domain)
dig -x 8.8.8.8

# Check DNS for a specific record
dig example.com MX
```

6 PUBLIC DNS SERVERS

Provider	IPv4 Address
Google DNS	8.8.8.8 8.8.4.4
Cloudflare DNS	1.1.1.1 1.0.0.1
OpenDNS	208.67.222.222 208.67.220.220

7 DNS CACHING

- DNS responses are cached by resolvers to reduce query time.
- Each record has a **TTL (Time To Live)**.
- Until TTL expires, cached data is used.



KEY TAKEAWAY

- ✓ DNS makes the Internet user-friendly.
- ✓ It translates domain names to IP addresses using a hierarchical system.
- ✓ Understanding DNS helps in troubleshooting connectivity issues.
- ✓ **Fast DNS = Fast browsing experience!**



13

HTTP & HTTPS

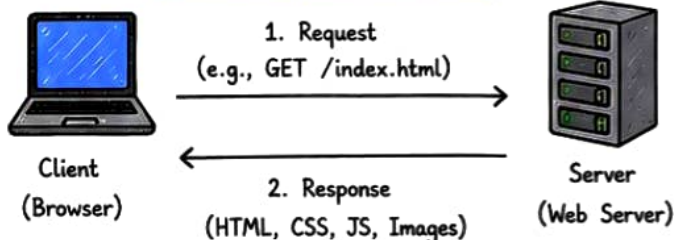
HTTP gets the job done,
HTTPS gets it done **securely**.
Always prefer **HTTPS**!



1 INTRODUCTION

- **HTTP** (HyperText Transfer Protocol) is the foundation of data communication on the Web.
- **HTTPS** (HTTP Secure) is the secure version of HTTP.
- HTTPS uses **SSL/TLS** to encrypt data between client and server.

HOW HTTP/HTTPS WORKS



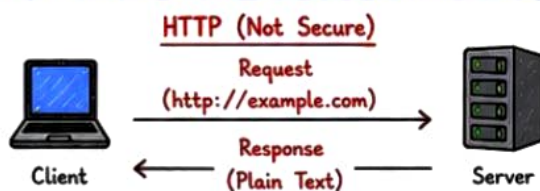
2 HTTP vs HTTPS

Feature	HTTP	HTTPS
Full Form	HyperText Transfer Protocol	HyperText Transfer Protocol Secure
Security	Not Secure	Secure (Uses SSL/TLS)
Port	80	443
URL	http://example.com	https://example.com
Data Encryption	No	Yes
Authentication	No	Yes (via SSL/TLS Cert)
SEO (Ranking)	Lower	Higher (Google prefers HTTPS)
Use Case	Internal / Test Environments	Websites, Applications, APIs (Always)

3 HTTP METHODS (COMMON)

Method	Purpose	Example
GET	Retrieve data from server	GET /index.html
POST	Send data to server	POST /login
PUT	Update existing data	PUT /user/1
DELETE	Delete data	DELETE /user/1
HEAD	Same as GET but returns headers only	HEAD /index.html
OPTIONS	Get communication options	OPTIONS /api
PATCH	Partial update	PATCH /user/1

4 HTTP vs HTTPS COMMUNICATION FLOW



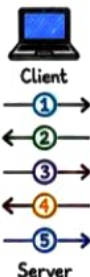
- Data is sent in plain text.
- Anyone can sniff and read the data.



- Data is encrypted using SSL/TLS.
- Safe from eavesdropping and tampering.

5 SSL/TLS HANDSHAKE (HIGH LEVEL)

1. Client Hello - Client requests secure connection.
2. Server Hello - Server responds with SSL certificate.
3. Certificate Verification - Client verifies certificate (CA, validity, domain).
4. Key Exchange - Both agree on session key.
5. Encrypted Communication - Secure data exchange begins.



6 COMMON HTTP STATUS CODES

Code	Meaning	Description
200	OK	Request successful
301	Moved Permanently	Resource moved permanently
404	Not Found	Resource not found
403	Forbidden	Access is denied
500	Internal Server Error	Something went wrong on server
503	Service Unavailable	Server is unavailable



KEY TAKEAWAY

- ✓ HTTP is simple but not secure.
- ✓ HTTPS encrypts data and protects user information.
- ✓ Use HTTPS for websites, APIs and sensitive applications.
- ✓ Security builds trust. **Always prefer HTTPS!**



14

TCP & UDP

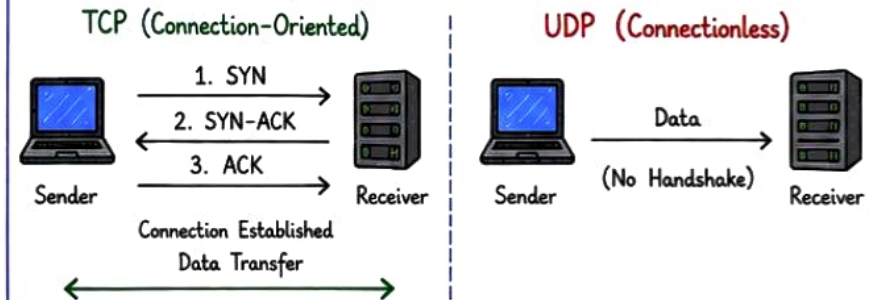
Both TCP and UDP work at the **Transport Layer** (Layer 4) of the OSI Model.



① OVERVIEW

- TCP is connection-oriented and reliable.
- UDP is **connectionless** and faster but **less reliable**.

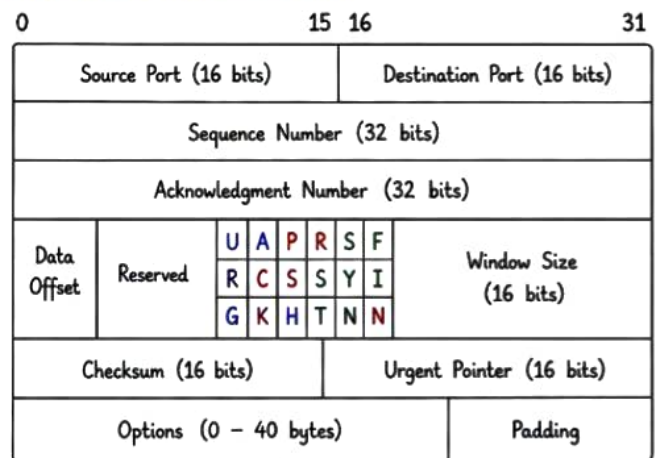
HOW THEY WORK



② TCP vs UDP COMPARISON

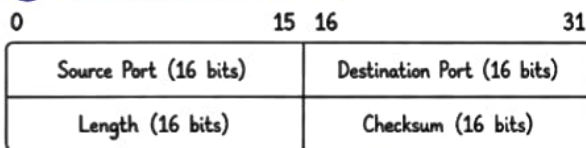
Feature	TCP	UDP
Connection	Connection-Oriented	Connectionless
Reliability	Reliable (ACK, Retransmission)	Not Reliable
Data Order	Maintains Order	No Order Guarantee
Speed	Slower	Faster
Overhead	Higher (Headers, Handshake)	Lower (Minimal Header)
Flow Control	Yes	No
Error Checking	Yes	Yes (Checksum only)
Use Cases	Web, Email, File Transfer, Database, SSH	Video Streaming, VoIP, Online Gaming, DNS
Header Size	20 - 60 bytes	8 bytes

③ TCP HEADER FORMAT



Flags: URG - Urgent, ACK - Acknowledgment, PSH - Push, RST - Reset, SYN - Synchronize, FIN - Finish

④ UDP HEADER FORMAT



- Simple and small header (only 8 bytes)
- Checksum is optional in IPv4, mandatory in IPv6
- No connection setup, start sending immediately

⑤ WHEN TO USE?

Use TCP when:

- ✓ You need reliable delivery
- ✓ Order of data is important
- ✓ Data integrity is critical
- ✓ Examples: Web (HTTP/HTTPS), Email, File Transfer, SSH

Use UDP when:

- ✓ Speed is more important
- ✓ Small data, real-time
- ✓ Some data loss is acceptable
- ✓ Examples: Video Streaming, VoIP, DNS, Online Gaming

⑥ REAL-WORLD EXAMPLES

Uses TCP



Uses UDP



KEY TAKEAWAY

- ✓ TCP = Reliable, Ordered, Connection-Oriented
- ✓ UDP = Fast, Lightweight, Connectionless
- ✓ Choose the right protocol based on your application needs.
- ✓ Understand the trade-off: Reliability vs **Speed**.

There is no "best" protocol, only the **right** protocol for the **right** job!

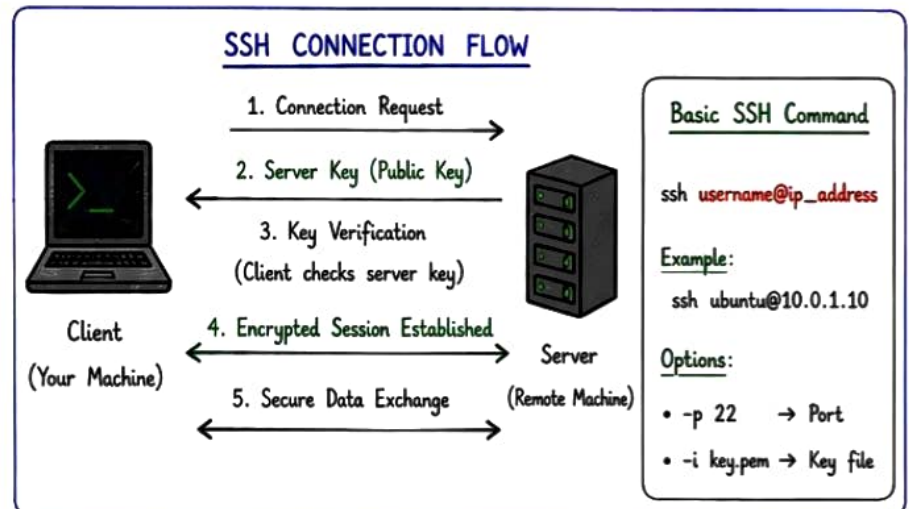


SSH = Secure Shell
SCP = Secure Copy



① SSH (Secure Shell)

- SSH is a network protocol that allows secure remote login.
- It **encrypts** all data including passwords and commands.
- Default port: **22**



② SCP (Secure Copy)

- SCP is used to securely copy files between local and remote systems.
- It uses SSH for data transfer.
- Default port: **22**

<u>SCP SYNTAX</u>		
Type	Syntax	Example
From Local to Remote	<code>scp file.txt username@ip:/path/</code>	<code>scp app.py ubuntu@10.0.1.10:/home/ubuntu/</code>
From Remote to Local	<code>scp username@ip:/path/file.txt /local/path/</code>	<code>scp ubuntu@10.0.1.10:/home/ubuntu/app.py ./</code>
Recursive Copy	<code>scp -r folder/ username@ip:/path/</code>	<code>scp -r myproject/ ubuntu@10.0.1.10:/backup/</code>

③ SSH KEY AUTHENTICATION (Better than Password)

1. Generate Key Pair

```
ssh-keygen -t rsa -b 4096
```

2. Copy Public Key to Server

```
ssh-copy-id username@ip
```

3. Login Without Password

```
ssh username@ip
```

Key Files

- `id_rsa` → Private Key
(Keep Secret!)
- `id_rsa.pub` → Public Key
(Share with server)

④ COMMON SSH CONFIG (~/.ssh/config)

Host myserver

`HostName 10.0.1.10`

`User ubuntu`

`Port 22`

`IdentityFile ~/.ssh/id_rsa`

Now you can just type: `ssh myserver`



KEY TAKEAWAY

- ✓ SSH provides secure remote access.
- ✓ SCP helps in secure file transfer.
- ✓ Use SSH Keys instead of passwords.
- ✓ Always verify server fingerprint before trusting.

Verify Server (First Time Login)

You will see something like:

ECDSA key fingerprint is

SHA256:ABcdEFGh...

Type **"yes"** to continue.

Stay calm.
Check → Identify → Fix
Verify → Document



① TROUBLESHOOTING METHODOLOGY

1. **Identify the Problem**
Understand the issue clearly.
2. **Gather Information**
Check logs, configs, metrics.
3. **Analyze**
Find root cause, not the symptom.
4. **Fix / Mitigate**
Apply the right fix carefully.
5. **Verify**
Confirm the issue is resolved.
6. **Document**
Write down what happened & solution.

② COMMON ISSUES & QUICK FIXES

Area	Common Issue	Possible Cause	Quick Checks / Fix
EC2 / SSH	Cannot SSH	Wrong key / SG / Instance stopped	Check SG (port 22) Check key & username Check instance status
Network	No Internet Access	Route table / NAT / IGW	Check route table Check NAT Gateway Check IGW attached
DNS	Name not resolving	VPC DNS / Route / Security	nslookup <domain> Check VPC DNS enabled Check SG (port 53)
RDS	Can't connect to DB	SG / Subnet / Credentials	Check SG (port 3306) Check DB endpoint Check username/password
S3	Access Denied	IAM / Bucket Policy	Check IAM permissions Check bucket policy Check object ACLs
Kubernetes	Pod not starting	Image / Resource / Config	kubectl describe pod <pod> Check logs: kubectl logs <pod> Check events
Jenkins	Build Failed	Code / Dependency / Config	Check console output Check credentials Check pipeline syntax

③ USEFUL LOGS & DEBUG COMMANDS

System Logs	journalctl -xeu <service> tail -f /var/log/syslog
SSH Debug	ssh -v user@ip
Network Debug	ping <ip> traceroute <ip>
Port Check	ss -tulnp grep <port> nc -zv <ip> <port>
DNS Check	nslookup <domain> dig <domain>
K8s Logs	kubectl logs <pod> -n <ns> kubectl describe <pod> -n <ns>
Service Status	systemctl status <service>

④ TOP COMMANDS BY CATEGORY

System Info	uname -a, df -h, free -h, uptime
Process	ps aux, top, htop, kill -9 <pid>
Disk	df -h, du -sh *, lsblk, iostat -xz 1
Network	ip a, ip r, ss -tulnp, netstat -tulnp
Security	ufw status, iptables -L -n -v, journalctl -u ssh
Docker	docker ps -a, docker logs <id>, docker system df
Kubernetes	kubectl get all -A, kubectl top nodes, kubectl get events --sort-by=metadata.creationTimestamp

⑤ CHECKLIST BEFORE ESCALATE

- ☒ Have you reproduced the issue?
- ☒ Checked logs & error messages?
- ☒ Checked configuration & permissions?
- ☒ Checked network connectivity?
- ☒ Any recent changes / deployments?
- ☒ Tried rollback or restart service?
- ☒ Documented findings & steps taken?

Escalate with clear details:

- What happened
- When
- Where
- What you tried
- Logs / Screenshots

⑥ COMMON ERROR CODES (QUICK REFERENCE)

404	Not Found	→ Resource / URL not found
401	Unauthorized	→ Invalid / Missing credentials
403	Forbidden	→ Access denied
500	Internal Server Error	→ Server side issue
502	Bad Gateway	→ Upstream service down
503	Service Unavailable	→ Service overloaded / Down
504	Gateway Timeout	→ Upstream timeout

KEY TAKEAWAY



- ☒ Troubleshooting is a skill - practice makes perfect.
- ☒ Always follow a systematic approach.
- ☒ Read logs carefully - they tell the real story.
- ☒ Fix the root cause, not just the symptom.
- ☒ Document and share knowledge for the team.

Good troubleshooting
saves time,
reduces downtime, and
builds trust!

